RADAR

Cari Risk Skorlama & Kredi Karar Destek Platformu

Teknik Dokümantasyon · v1.0

DynamicAI · 2026

Radar — A'dan Z'ye Teknik Dokümantasyon

🎯 Şirketinize Özelleştirilebilir

Radar açık kaynak (AGPL-3.0) bir platformdur ve **her şirketin kendi iş kuralları, sektör profili ve risk iştahına göre özelleştirilebilir**. Skorlama formülleri, ağırlıklar, sektör risk faktörleri, makro varsayımlar (faiz, enflasyon), vade kademeleri ve karar mantığı tamamen yapılandırılabilir.

Kurumsal entegrasyon, özel modül geliştirme, on-premise deployment ve eğitim hizmetleri için DynamicAI ile iletişime geçebilirsiniz: *osmanedembele6361@gmail.com*

> Cari risk skorlama ve kredi karar destek platformu. Son güncelleme: 2026-05-02

1. Proje Özeti

Radar, KOBİ ve kurumsal firmalara yönelik bir **kredi skorlama / cari risk değerlendirme** uygulamasıdır. Findeks benzeri bir derecelendirme servisinin firma-içi (private) muadili olarak konumlanır.

Temel iş akışı: 1. Kullanıcı müşteri (cari) açar 2. Yaşlandırma (aging) ve kredi talebi verisi girer veya Excel'den toplu içe aktarır 3. Sistem; ödeme disiplini, finansal rasyolar (Z-Score, F-Score, DSCR, ICR), sektör risk faktörleri ve makro koşullar üzerinden bir **kredi notu** üretir 4. Monte Carlo simülasyonu + isimli stres testleri ile senaryo dağılımı hesaplanır 5. Çok dilli (TR/EN/ES/DE) detaylı PDF/HTML rapor sunulur

2. Teknoloji Yığını

Katman	Teknoloji
Backend	Python 3.11, **Flask 3.0** (Blueprints)
ORM	**SQLAlchemy 2.0** (scoped_session)
DB	PostgreSQL (prod) / SQLite (dev) — dual target
Auth	flask-login + **Authlib** (Google OAuth) + itsdangerous (mail verify)
Mail	flask-mail (SMTP)
i18n	**flask-babel 4.x** (locale_selector pattern)
Templating	Jinja2 + Bootstrap 5.3 + Bootstrap Icons
API Docs	**Flasgger** (OpenAPI/Swagger)
Async	Celery + Redis (planlı, henüz aktif değil)
Container	Docker + docker-compose, `network_mode: host`
WSGI	gunicorn 4 worker
Reverse Proxy	nginx — `technodai.com/radar/` → `127.0.0.1:5001`
Excel	pandas + openpyxl

3. Mimari (Yüksek Seviye)

```
┌──────────────────────────────────────────────────────────┐
│  Browser (TR/EN/ES/DE)                                   │
└──────────────────────────────────────────────────────────┘
                        │
                        │ https://technodai.com/radar
                        ▼
                  ┌──────────┐
                  │  nginx   │  TLS, prefix routing
                  └──────────┘
                       │
                       │ proxy_pass :5001
                       ▼
              ┌──────────────────┐
              │  gunicorn × 4    │  Docker (host net)
              │   ↓              │
              │  Flask app       │
              │   ├ blueprints/  │  auth, main, customer,
              │   │              │  scoring, admin, api
              │   ├ enterprise/  │  i18n, RBAC, audit, webhooks
              │   └ analytics/   │
              └──────────────────┘
                        │
                        │
            ┌───────────┼──────────┐
            ▼           ▼          ▼
       PostgreSQL   SMTP      Google OAuth
       (advisory   (verify     (login)
        locks)     mails)
```

---

4. Modül Haritası

4.1 Çekirdek

Dosya	Sorumluluk
`app.py`	Flask factory (`create_app`), blueprint kayıt, `init_db()` , enterprise feature init
`database.py`	SQLAlchemy modeller, dual-DB engine, lightweight ALTER migration, **Postgres advisory lock** (concurrent worker safety)
`extensions.py`	`login_manager` , `mail` , `ts` (URLSafeTimedSerializer) — modül seviyesinde paylaşılan singleton'lar
`credit_scoring.py`	`CreditScorer` sınıfı — skor hesaplama, expert assessment notları, Monte Carlo + isimli stres testleri
`aging_analyzer.py`	Yaşlandırma kayıtlarından gecikme metriği üretme
`excel_import.py`	Toplu müşteri/kayıt import (pandas)
`currency.py`	TL formatlama, FX dönüşüm yardımcıları
`logger.py`	Yapılandırılmış logging
`translations.py`	4 dil sözlüğü (TR/EN/ES/DE) — düz dict, ~70 kB

4.2 Routes (Blueprints)

Blueprint	Path	Sorumluluk
`auth`	`/login` , `/register` , `/logout` , `/google-login` , `/verify/<token>`	Kullanıcı kimlik doğrulama
`main`	`/` , `/nedir` , `/guvenlik` , `/set_language/<lang>`	Anasayfa, statik içerik, dil değişimi
`customer`	`/musteri/...`	CRUD + detay
`scoring`	`/skor/...` , `/rapor/<id>`	Skor hesaplama, rapor render
`admin`	`/admin/...`	Ayarlar, kullanıcı yönetimi, audit log
`api`	`/api/v1/...`	REST endpoint'ler (Flasgger ile dokümante)

4.3 Enterprise Katmanı (Phase 8-11)

Dosya	Sorumluluk
`enterprise.py`	`init_enterprise_features(app)` — RBAC, audit, multi-tenant init
`i18n_utils.py`	flask-babel 4.x locale selector
`analytics.py`	Tenant-level metrik toplama
`api_docs.py`	Flasgger Swagger UI yapılandırması
`webhooks.py`	Outbound webhook gönderimi (Slack, Discord, custom)

---

5. Veri Modeli

```
Tenant 1──* User 1──* Customer 1──* AgingRecord
                                  │
                                  ├──* CreditRequest
                                  └──* CreditScore (history)

User *──* Role  (UserRole pivot)
Tenant 1──* AuditLog
```

Önemli: - `tenant_id` her tablo için `nullable=True default=1` — eski kayıtlar default tenant'a atanır - `is_sample=True` müşteriler tüm kullanıcılara görünür (demo data) - `CreditScore` versiyonludur — her hesaplama yeni bir kayıt yaratır, geçmiş silinmez

---

6. Deployment Mimarisi

Prod (technodai.com)

```
~/apps/radar/                  # git clone
├── docker-compose.yml         # local-only port binding (5001)
├── .env                       # SECRET_KEY, DB_URL, OAuth, SMTP
└── data/                      # SQLite fallback + uploads (volume)
```

- Container `radar_v1_app`, restart: always, host network
- Postgres `radar` DB, advisory lock (`pg_advisory_lock(7382001)`) ile 4 worker init race'i önlenir
- nginx vhost: `/radar/` prefix `proxy_pass http://127.0.0.1:5001/`

Lokal (dev)

- Aynı compose file (Desktop symlink), port `8005`
- SQLite fallback ile DB-siz çalışabilir

Dağıtım Akışı

```
lokal düzenleme
   ↓ git commit + push
   ↓ ssh prod
cd ~/apps/radar && git pull
sudo docker compose up -d --build
   ↓ smoke test
curl -I https://technodai.com/radar/
```

---

7. Geliştirme Workflow'u

1. **Branch**: `main` üzerinde küçük commit'ler. Büyük feature'lar için topic branch.
2. **Migration**: `database.py:_run_lightweight_migrations()` içinde `ALTER TABLE IF NOT EXISTS` paterni — Alembic yok (kasıtlı, küçük şema için aşırı).
3. **Seed**: `seed_sample_data.py` — yeni kurulumda demo müşteri seti.
4. **Test**: `tests/integrity_check.py` (manuel) — birim test henüz yok.
5. **i18n**: Yeni metin ekleyince `translations.py` 4 blokta da güncellenmeli. CI henüz key parity check'i yapmıyor (eklenebilir).
6. **Commit mesajı**: `<type>(<scope>): <özet>` — `feat`, `fix`, `chore`, `refactor`.

---

8. Kullanım Akışı (Son Kullanıcı)

Adım	Sayfa	Aksiyon
1	`/register`	E-posta/şifre veya Google ile kayıt → mail doğrulama
2	`/`	Müşteri listesi — yeni müşteri ekle
3	`/musteri/yeni`	Cari bilgileri (sektör, özkaynak, ciro vb.)
4	`/musteri/<id>`	Yaşlandırma satırı + kredi talebi gir
5	`/musteri/<id>/import`	Excel ile toplu içe aktar
6	`/skor/<id>`	Skor hesapla — admin ayarları (faiz, enflasyon) uygulanır
7	`/rapor/<id>`	Detaylı rapor — yönetici değerlendirmesi, senaryolar, isimli stres testleri
8	`/admin`	Ayarlar, kullanıcı yönetimi, audit log

---

9. Yol Haritası — Eklenecekler

9.1 Kısa Vade (1-2 hafta)

- [ ] **Vade modülü genişletme**: Şu an statik kademeli vade (`vade_15..90`); ödeme disiplini bandına göre dinamik hesaplama (DSO + sektör mediyanı)
- [ ] **Translations CI guard**: 4 dil arası key parity test (eksik anahtar = build fail)
- [ ] **PDF export**: Rapor `?format=pdf` için WeasyPrint entegrasyonu
- [ ] **Birim test iskeleti**: `pytest` + `pytest-flask` , en az `credit_scoring` için %70 coverage
- [ ] **Audit log UI**: Mevcut tablo görüntülenmiyor — admin panele filtrelenebilir liste

9.2 Orta Vade (1-2 ay)

- [ ] **Celery aktivasyon**: Excel import + skor batch'i async kuyruğa al (büyük dosyalarda timeout problemi var)
- [ ] **Webhook event genişletme**: `score.calculated` , `customer.high_risk` , `request.spike` event'leri
- [ ] **Rate limiting**: `flask-limiter` — `/api/v1/*` per-tenant kota
- [ ] **Sektör verisi enjeksiyonu**: Halka açık sektör rasyo benchmark'ı (KGK / TÜİK / Findeks rapor) — şu an sadece peer median
- [ ] **Notification channels**: E-posta + Slack + Discord — yüksek risk alarmları
- [ ] **Multi-currency**: Şu an her şey TL — USD/EUR cari için FX snapshot tablosu

9.3 Uzun Vade (3+ ay)

- [ ] **ML katmanı**: Mevcut kural-tabanlı skor + opsiyonel logistic regression / gradient boosting overlay (kayıt sayısı yeterli olduğunda)
- [ ] **Open Banking**: Hesap hareketi pull → otomatik aging
- [ ] **Mobile app**: PWA veya React Native — okuma odaklı (skor görüntüleme, alarm)
- [ ] **Marketplace tenant onboarding**: Self-serve SaaS (şu an manuel tenant açma)
- [ ] **GDPR/KVKK paneli**: Veri ihracı, silme talebi, retention policy
- [ ] **2FA**: TOTP (pyotp) — özellikle admin rolleri için
- [ ] **Audit log export**: SIEM (Splunk, ELK) için JSON akışı

---

10. Teknik Borç

Konu	Etki	Çözüm
Alembic yok	Şema değişimi riskli	Lightweight ALTER yeterli kalıyor; karmaşıklaşırsa Alembic'e geç
Test coverage < %5	Refactor riskli	Pytest iskeleti kur, kritik path'lere odaklan
`translations.py` tek dosya 70 kB	Merge conflict mıknatısı	Dil başına ayrı dosya + lazy load
Authlib deprecation warning (`authlib.jose`)	Şu an çalışıyor, 2.0'da kırılacak	`joserfc` 'e geç
Inline SVG ikonlar	Bakımı zor	Bootstrap Icons + custom font birleştir
`docker-compose.yml` `version` field obsolete	Sadece warning	Sil
Sunucudaki `~/apps/radar/docker-compose.yml` git'le çakışıyor (port farkı)	Her pull'da stash gerekiyor	`docker-compose.override.yml` ile ayrıştır
Webhook retry yok	Tek deneme — başarısız olursa kayıp	Celery retry policy
Loglar JSON ama ID korelasyonu yok	Trace zor	request_id middleware

---

11. Güvenlik Notları

- ✅ Secret/PAT'lar `.env`'de, repoda yok (`.gitignore` korunuyor)
- ✅ Mail verify zorunlu (silent ts=None bug'ı düzeltildi)
- ✅ Google OAuth state validation Authlib'in kendisinde
- ✅ Postgres advisory lock — concurrent init race koruması
- ✅ `is_sample` flag tenant izolasyonunu bypass etmiyor (sadece okuma)
- ⚠️ CSRF token: form'larda manuel yok, flask-wtf eklenmeli
- ⚠️ Rate limit yok — bruteforce login açık
- ⚠️ Audit log delete = soft delete değil, fiziksel silme (compliance riski)
- ⚠️ DB backup automation yok — manuel `pg_dump`

---

12. Operasyonel Komutlar

```
# Lokal dev
python app.py                                    # SQLite ile

# Docker (lokal port 8005)
docker compose up -d --build
docker compose logs -f radar-app

# Prod deploy (lokal makineden)
sshpass -p "$SERVER_PASS" ssh "$SERVER_USER@$SERVER_IP" \
  "cd ~/apps/radar && git pull && \
   echo \$PASS | sudo -S docker compose up -d --build"

# Smoke test
curl -I https://technodai.com/radar/

# DB backup (prod'da)
sudo docker exec radar_v1_app pg_dump -U radar radar > backup_$(date +%F).sql

# Migration (lightweight, otomatik init_db'de çalışır)
python -c "from database import init_db; init_db()"

# Seed
python seed_sample_data.py

# Translation key parity (manuel)
python -c "from translations import translations; \
  ks = {l: set(translations[l]['expert_assessments'].keys()) for l in translations
  print('diff:', ks['tr'] ^ ks['en'])"
```

---

13. Karar Kayıtları (ADR-mini)

1. **Alembic yerine lightweight ALTER**: Şema küçük, downtime kabul edilebilir. Karmaşıklık artarsa revize edilir.
2. `network_mode: host` : nginx ile aynı host'ta, port mapping karmaşıklığı yok. Multi-host'a geçerken değişir.
3. **flask-babel 4.x**: Flask 3.0 ile 3.x uyumsuz (`locked_cached_property` kaldırıldı).
4. **Tek tenant default ID=1**: Eski kayıtlar geriye dönük taşındı; multi-tenant'a geçiş kesintisiz.
5. **Postgres advisory lock anahtarı 7382001**: Sabit magic number — tüm worker'lar aynı kilidi alır, ilki tablo yaratır, diğerleri bekler.
6. **Sponsor pill footer'da**: Navbar'da yer kalmıyordu, görünürlük düşmesin diye footer'a alındı.

---

14. Lisans & Açık Kaynak

- **Proje lisansı:** AGPL-3.0 — host edip hizmet sunan herkes kaynak kodu paylaşmak zorunda
- **Bağımlılıklar:** Tümü permissive (MIT/BSD/Apache); GPL/AGPL yok → AGPL ile uyumlu
- **Dikkat çeken bağımlılıklar:**
- `psycopg2-binary` (LGPL) — dynamic linking, sorun yok
- `certifi` (MPL-2.0) — dosya-bazlı copyleft, dokunmadığımız sürece OK
- **Lisans denetimi:** `pip-licenses` ile yapıldı, çıktı temiz

15. Test & Kalite

Mevcut Testler

- `tests/security_smoke.py` — 8 güvenlik kontrolü:
- `.env` ve sırlar git'te değil
- Özel anahtar dosyaları yok
- Hardcoded SECRET/PASSWORD/API_KEY yok
- Kritik paketler pinli (flask, sqlalchemy, authlib, itsdangerous, flasgger)
- Tüm protected route'lar `@login_required` ile sarılı
- SECRET_KEY set ve uzun
- LICENSE dosyası AGPL-3.0
- `tests/integrity_check.py` — manuel skor doğrulama (DB gerektirir)

CI (GitHub Actions)

- `.github/workflows/ci.yml` — her push/PR'da:
- pytest security suite
- `pip-audit` ile bağımlılık güvenlik taraması
- Translation key parity (TR/EN/ES/DE arası eşitlik)

Çalıştırma

```
pytest tests/ -v                    # tüm testler
pytest -m security                  # sadece güvenlik smoke
pip-audit                           # CVE taraması
```

16. Referanslar

- Repo: `git@github.com:Dai-Solutions/Radar.git`
- Prod: https://technodai.com/radar/
- Sponsor: OpenCollective + Patreon (`https://www.patreon.com/c/DynamicAI`)
- Findeks (rakip): https://www.findeks.com/